



Pacing for Mastery: Optimizing LLM Interactions for Learning

Karena Tran
University of California, Irvine
Irvine, CA, USA
karenast@uci.edu

Ge Gao
University of California, Irvine
Irvine, CA, USA
gaog5@uci.edu

Angela Lombard
University of California, Irvine
Irvine, CA, USA
lombara1@uci.edu

Tyler Yu
University of California, Irvine
Irvine, CA, USA
tyleryy@uci.edu

Haoning Jiang
University of California, Irvine
Irvine, CA, USA
haoningj@uci.edu

Thomas Y. Yeh
University of California, Irvine
Irvine, CA, USA
tomyeh@uci.edu

Abstract

Large Language Models (LLMs) hold significant potential for transforming computer science education, yet concerns over their possible negative effects on student learning and retention have slowed broader instructor adoption. Evidence on LLM use is mixed. While novices may benefit from the generative capabilities of AI, they also risk developing overreliance. To address these concerns, we investigate how the pace of interaction with AI assistants affects learning in introductory CS courses by deploying three AI assistants (Fast, Medium, and Slow) in a classroom setting. Our results show that the slower-paced, Socratic-style AI assistant significantly increases learning, especially for students with less prior knowledge. Although faster-paced interaction benefits more advanced students initially, learning retention degrades enough to negate those gains. Surprisingly, the medium-paced assistant with typical instructor preprompt elements shows no statistically significant improvements. Given that students may use fast-paced commercial AI tools for coursework regardless of policy, offering a slower-paced, Socratic-style AI alternative could meaningfully improve overall student learning outcomes.

CCS Concepts

- **Applied computing** → **Interactive learning environments;**
- **Social and professional topics** → **Computing education;**
- **General and reference** → **Empirical studies.**

Keywords

interaction pace, learning, retention, novice programmers, CS1, LLMs, Generative AI, AI assistants, classroom

ACM Reference Format:

Karena Tran, Ge Gao, Angela Lombard, Tyler Yu, Haoning Jiang, and Thomas Y. Yeh. 2026. Pacing for Mastery: Optimizing LLM Interactions for Learning. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1 (SIGCSE TS 2026)*, February 18–21, 2026, St. Louis, MO, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770762.3772501>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCSE TS 2026, St. Louis, MO, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2256-1/2026/02
<https://doi.org/10.1145/3770762.3772501>

1 Introduction

The rising adoption of LLMs by CS students has quickly shifted the question from *whether* to incorporate to *when* and *how* to integrate LLMs. Students show a growing preference for using LLMs in CS courses, especially if they are endorsed by the course instructor [22, 27]. While AI tools have the potential to improve learning outcomes, their effectiveness largely depends on how students use them. Common help-seeking patterns include debugging and getting started on assignments, but student queries often contain insufficient information [20, 29], resulting in low-quality responses [19, 23]. Additionally, over-reliance on generative AI has become a primary concern for instructors [25]. Access to immediate solutions hinders the learning process [10, 26, 30], and students tend to passively accept without understanding LLM outputs [11].

To address these challenges, we propose enhancing engagement with LLMs by increasing interaction through effective questioning and redirection. This is motivated by the observation that LLMs enhance learning the most when students attempt problems on their own before receiving explanations [13]. Prior work pointed out benefits of intentionally slowing down LLM interactions such as aligning users' perceived and actual coding ability [11], promoting self learning [12], enabling independent reflection [31], and learning from interactivity [35]. Our work builds upon this body of work by seamlessly integrating redirection and effective questioning.

We evaluated the effects of AI assistant interaction pace by implementing three models (Fast, Medium, and Slow) and deploying them to students over two consecutive terms of an introductory CS course. There were 132 students in the first term, and 102 students in the second term. To our knowledge, this is the first study to evaluate the impact of LLM interaction pace on student learning within a CS classroom setting. Our research questions are the following:

RQ1: How does the pace of interaction when using Generative AI assistants impact student learning?

RQ2: How does the pace of interaction when using Generative AI assistants impact student learning retention?

2 Related Work

Generative AI tools are increasingly popular among students and instructors, yet their impact on student learning remains unclear. In a controlled study, Kazemitabaar et al. [10] examined how novices learn Python using Codex, OpenAI's programming LLM. Although novices supported by Codex initially outperformed those without

AI on code-authoring tasks, they ultimately made more errors in a subsequent retention test.

A primary concern of generative AI for educators revolves around overreliance [11, 25, 36]. Research indicates that novices often accept LLM outputs passively, failing to verify or fully understand the solutions provided [5, 16]. One critical factor is the ability to prompt LLMs effectively. Nguyen et al. [23] showed that novices encounter difficulties generating workable code for problems at their skill level, partly due to insufficient guidance in formulating prompts. Lucchetti et al. [19] explored this issue further, concluding that novices struggle primarily because their prompts lack the vital information necessary for LLMs to generate functional solutions.

To address these shortcomings, researchers have introduced design modifications tailored to educational contexts. CodeHelp [14] employed guardrails to prevent solution generation and to focus on explanations and scaffolding. Similarly, Liu et al. [17] created CS50 Duck, an AI chatbot underpinned by ChatGPT [24] adapted with course-specific guidelines. These tools maintained conversational support yet occasionally offered responses deemed “overly helpful,” risking the creation of complete solutions outright [18]. To mitigate this, Liu et al. [18] deployed conversational strategies such as clarifying questions through few-shot prompting and fine-tuning. While effective in principle, direct insight from novice learners on these methods has been limited.

Employing Socratic questioning by AI assistants can be conceptualized as a form of co-regulation [9], whereby the assistant’s guiding questions and strategic redirection constructively slow down the interaction. This shift promotes self-regulation among learners, who must reflect on and articulate their own reasoning before receiving further assistance. Yeh et al. [35] observed that interactive LLMs could enhance novices’ ability to prompt effectively and facilitate a deeper understanding of how to interact with AI tools. Building on this, our study investigates how deliberately paced LLM interactions driven by Socratic-style questioning affect student learning in an introductory computer science course. We assess both immediate learning gains and retention, aiming to clarify the extent to which “slowing down” the LLM can foster more robust self-regulation and long-term mastery.

3 Methodology

The initial data for this study was collected from a CS1 course during the Fall 2024 term at a large public research university in the United States. This research was conducted as a longitudinal study embedded within the course. All 132 of the students enrolled at the end of the term consented to participating in the study. Informed consent was obtained through a pre-survey with a link to the study information sheet and emphasized voluntary participation.

To verify our findings, we duplicated the study in the same course for the Winter 2025 term. 102 students participated under informed consent. The AI models, instructor, labs, and assessments remained the same. First, we analyzed the data from the two terms as separate studies. Then, we conducted a combined analysis to increase statistical power while accounting for the quarter as a factor.

3.1 AI Assistant Designs

For our study, *interaction pace* refers to the speed at which the AI assistant moves the user toward a solution. A faster pace means the user performs less cognitive work, whereas a slower pace requires the user to exert more effort in figuring things out. A faster paced AI responds immediately with detailed answers, encouraging students to offload critical thinking. A slower paced AI, on the other hand, engages students more gradually with redirections, hints and questions to guide them through their own reasoning processes. To evaluate the effects of interaction pace on student learning, we created three different AI assistants, all based on the same LLM.

3.1.1 Fast. The baseline model uses OpenAI’s GPT-4o LLM, which has been released to all students and faculty at the study institution. This model, representative of commercial LLMs with no guardrails, has the fastest interaction pace. When a student requests debugging help or asks for a solution (one-shot prompting [28]), the Fast model is more than willing to provide the solution along with explanations. We observed the same behavior when we prompted various leading commercial LLMs such as GPT-4.1, O1, O3-mini, Claude Sonnet 4, Claude Sonnet 3.5 and Mistral Large 2.

3.1.2 Medium and Slow. We implemented two interactive AI assistants using OpenAI’s GPT-4o LLM. They create an interactive experience by asking users guiding questions. To achieve this, we created Medium and Slow pre-prompts with the following:

- A description of the role of the system as an AI tutor. Learning objectives and course-specific information are provided (e.g. programming language and relevant libraries).
- Instructions to categorize the student’s initial prompts into one of five use cases (problem decomposition, debugging, concept explanation, code generation, code explanation), then determine how to help the student based on the use case.
- Instructions to ask questions for clarifications and for guiding students towards self-learning.
- Instructions to prevent code generation.

The Slow paced pre-prompt includes additional instructions to ask one question at a time and to emphasize guided self-learning. The Slow model employs a Socratic questioning style [32, 33] and uses redirection as an instructional strategy, prompting students to work through problems independently before receiving hints. This approach fosters deeper problem-solving skills and a stronger sense of ownership in learning. The Slow model consistently redirects students who request answers with one-shot prompting [28]. Figure 1 shows example AI responses to the same user prompt.

3.1.3 Quantifying Interaction Pace. We characterize interaction pace according to the following factors: (1) the frequency of questions within LLM responses, (2) the number of turns before the LLM provides a verbose response, (3) the overall amount of information provided by the LLM, and (4) the frequency of redirection. A turn is a user prompt followed by a model response. Figure 2 illustrates the differences between the models.

Commercial LLMs tend to project confidence by providing large blocks of information without posing questions. The average number of words in a AI response is 281 for Fast, 88 for Medium, and 87 for Slow. We label any response exceeding 100 words as a verbose

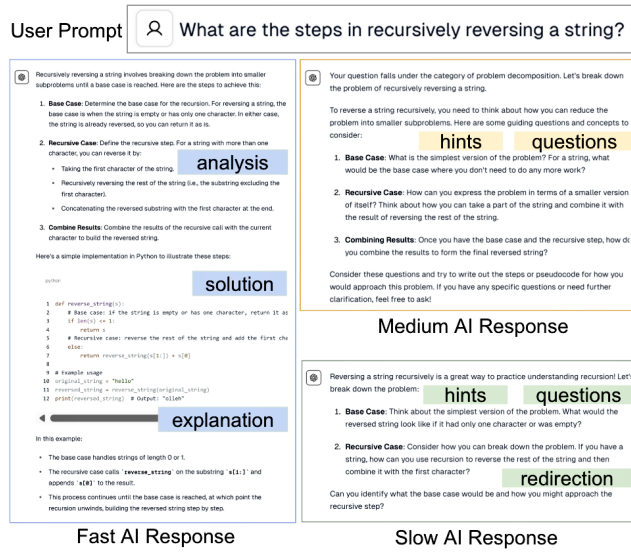


Figure 1: Example Interactions with AI Assistants

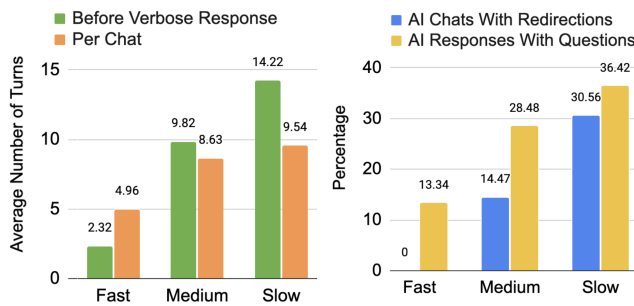


Figure 2: Characterizing AI Assistants

response. The trend shown in Figure 2 remains consistent with thresholds of 150 or 200 words. Commercial LLMs are also quick to provide answers, even when students do not ask for solutions. In contrast, the interactive models redirect students to think about the assignment or error before providing information and hints.

As shown in Figure 2, the Fast model has the fewest questions, the lowest turns per chat, the fewest turns before a verbose response, and zero redirects. By contrast, the Slow model emphasizes engagement by redirecting, questioning and limiting verbosity: 31% of its chats include redirection, 36% of its responses contain questions, it has the most turns per chat, and it averages 14 turns before a verbose response. The Medium model occupies a middle ground between these two extremes.

3.2 Experimental design

3.2.1 Measuring Learning. The experiments conducted to measure learning occurred during eight 80-minute structured lab sessions. Each lab included a programming assignment focused on a specific concept previously covered in lecture. With access to the assigned AI assistant, teaching assistants (TAs), and learning assistants (LAs),

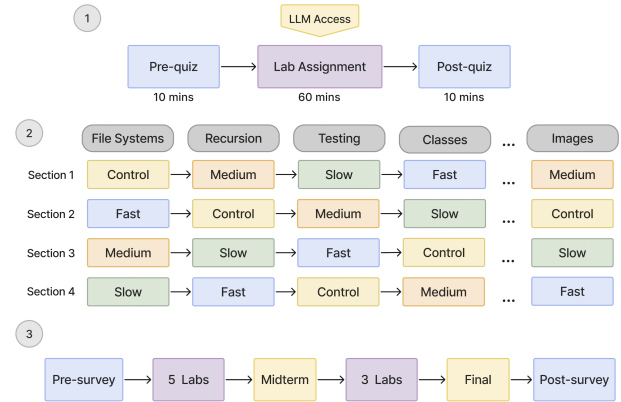


Figure 3: Experimental setup: (1) workflow for labs. (2) model assignments per section across topics. (3) course structure.

students worked on the assignment in the provided lab classroom. At the start of a structured lab, students spend 10 minutes to take the pre-quiz. Then, the students worked on the programming assignment for 60 minutes. In the last 10 minutes, students took a post-quiz to measure their learning gain. Quizzes were administered using a lock-down browser along with TA proctoring. TAs took attendance during lab and monitored for any external AI usage.

This study is set up as a replicated Latin Square design as shown in Part 2 of Figure 3. Students were divided into four groups based on their lab enrollment. For each lab, each group was assigned one of four experimental conditions: no access to an AI assistant (Control), Fast model, Medium model, and Slow model. For the Medium and Slow models, the learning objectives were updated according to the lab topic. The model assignments rotated between lab groups across different labs, ensuring that all groups experienced each condition an equal number of times during the term.

3.2.2 Pre and Post Lab Quiz Questions. The pre and post quizzes consisted of six multiple choice questions, each with five choices. Three of the questions tested higher-level cognitive skills, and the other three tested lower-level cognitive skills. The questions were designed to cover the first 5 levels of Bloom's Taxonomy [1, 7], and they were randomly selected from a question bank with choices shuffled between students. Example questions are shown in Table 1.

3.2.3 Measuring Retention. To study learning retention, we designed each exam question to cover a single lab topic. We measured retention by comparing the post-quiz score to the corresponding exam question score. Part 3 of Figure 3 shows the course structure. For the exams, each multi-part question focused on one lab topic. The midterm covered programming concepts from the first five labs, and the final covered domain-specific concepts from the last three labs. Example exam questions are provided in Table 1.

3.2.4 Survey data. At the beginning of the term, students were asked to complete a pre-study survey that collected prior programming experience and demographic information. At the conclusion of the study, students were invited to complete an anonymous post-study survey with 5-point Likert scale and free-response items asking about their experiences with the provided AI Assistants.

ID	Quiz Questions
1	Topic: Recursion Bloom's Level 3: Applying Question: Complete the following recursive function to reverse a string:
2	Topic: Exception Bloom's Level 4: Analyzing Question: What is a potential risk of allowing a program to continue after catching an exception?
3	Topic: Exceptions Bloom's Level 5: Evaluating Question: If your program may write to a file without the correct permission, what's the best way to prevent it from crashing?
ID	Exam Questions
4	Topic: Recursion Bloom's Level 3: Applying Question: What is the output of the following recursive function call?
5	Topic: File System Bloom's Level 4: Analyzing Question: The <code>print_path</code> function takes a path and recursively lists all files and directories under the provided path. Identify the error and provide the line number(s). Then, provide the fixed code for this function.
6	Topic: Recursion Bloom's Level 6: Creating Question: Implement a recursive function called <code>search</code> that takes two arguments (<code>world</code> and <code>target</code>). <code>World</code> is a nested list, and <code>target</code> is an integer. The function should return the depth level if the target character is found within the nested list; otherwise, it should return <code>-1</code> .

Table 1: Example Quiz and Exam Questions

Follow these requirements when implementing your note-taking program:

1. It must store notes in a file called `pynote.txt`. The program must create this file if it does not already exist.
2. The `pynote.txt` file must be located in the same directory as your `lab1.py` module.
3. The program should print the welcome message, file location, and all existing notes stored in the `pynote.txt` file when it starts as shown below.
4. The program must continue accepting notes from a user until the user enters the letter "q".
5. The program must work on both Windows and POSIX systems.
6. The program must contain the following functions.
 1. `start_notes` function takes 0 parameters and returns a 2-element tuple. The first element of the tuple is a string which holds all the output before asking the user for an input. This includes the welcome message, file location, and file contents as shown below. The second element of the tuple is a Path object created using the Path class from the `pathlib` module. It represents the file system path to a file named `pynote.txt` located in the current working directory.
 2. `write_notes` function which takes a string, the file path, as input. Inside this function, ask the user for the notes to add to the `pynote.txt` file. It writes the string into the `pynote.txt` file, separating each unique note with a line of space.
 3. `main` function uses the previous 2 functions to implement the entire program as shown below.
7. Write the program such that it can be imported as a module.

Figure 4: Example Lab Assignment

3.3 Data Collection

To ensure valid data for our analysis, we post-processed the collected data based on the following criteria. When measuring learning gain, if a student was absent or missed either the pre-quiz or post-quiz scores for a particular lab, we excluded both quiz scores for that lab. Similarly, when measuring retention, if a student was missing the post-quiz score for a particular lab, we removed the scores for the lab and the corresponding exam question.

3.4 Analysis Method

We conducted statistical tests at a significance level of $\alpha = 0.05$ to determine whether the observed differences in response measures are statistically significant. To reflect real-world conditions, minimize bias from noncompliance, and preserve the benefits of randomization, we applied intention-to-treat (ITT) analysis [21] to the data. This method included all participants in their originally assigned groups, regardless of adherence to the actual usage. For context, we provide the AI assistant usage information in Table 2.

We employed a mixed-effects linear regression model (MLM) [4, 8] to explore differences in learning gain and learning retention

	F24 Prompts	F24 UU	W25 Prompts	W25 UU
Total	–	52%	–	81%
Fast	25%	52%	29%	58%
Med	37%	37%	30%	48%
Slow	38%	32%	41%	63%

Table 2: Percentage of Prompts and Unique Users in Class.

across the experimental groups. MLM is well-suited because it handles repeated measurements from students across labs with varied AI paces, accommodates the hierarchical structure of data nested within students and lab sections, deals with missed quizzes, and accounts for potential time-varying effects. MLM primarily depends on the distribution of residuals rather than strict linearity in the outcome. Based on the QQ and residual vs fitted values plots, the residuals appear approximately normal and do not show a strong pattern, indicating that the normality assumption is satisfied for MLM. The model is specified as:

$$Y_{ij} = \beta_0 + \beta_1 \text{Model}_{ij} + u_i + v_j + \epsilon_{ij}$$

Y_{ij} is the learning gain or learning retention for student i for lab session j . β_0 is the fixed intercept representing the Control. β_1 is the coefficient for the fixed effect of the AI assistant (*Model*). Model_{ij} indicates the AI assistant assigned to student i in lab session j . u_i is the random effect for student i , accounting for individual deviations from the average gain. v_j is the variance component for lab number j , allowing for different variances across lab sessions. ϵ_{ij} is the residual error term for student i in lab session j .

3.4.1 Experimental Setup Summary. There are 8 lab assignments and 4 lab sections. For each lab assignment, every lab section is assigned one of four AI paces (Control, Fast, Medium, or Slow). As shown in Figure 3, the pace assignment rotates so that each lab section experiences every pace an equal number of times. All students in a section are assigned the same pace for a given lab, and all student work under multiple paces over the course.

3.4.2 Institutional Review Board. The study was considered exempt from the institution's Institutional Review Board review as the research was conducted in an educational setting involving normal educational practices and only aggregated data is reported.

4 Results and Discussion

4.1 Impact on Student Learning (RQ1)

To investigate how AI interaction pace influences learning, we compared students using AI assistants with a control group receiving no AI. We measured learning using relative gain, which captures proportional improvement from a student's baseline, and normalized gain, which indicates how much of the possible improvement was achieved while accounting for extreme pretest scores. Normalized gain is widely used in STEM education to gauge instructional effectiveness. Hake [6] classified normalized gain as low (<0.3), medium (0.3–0.7), or high (>0.7) when evaluating an entire course.

4.1.1 Learning Gain. We computed both relative and normalized learning gains for each of the eight labs and performed separate MLM analyses on each metric. We examined three datasets (Fall 2024, Winter 2025, Combined) while treating each per-lab learning

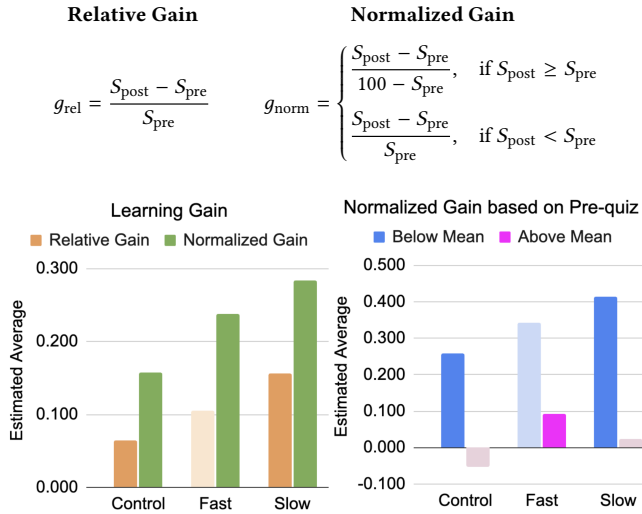


Figure 5: Learning Gain for Different Interaction Pace.

	Relative Gain		Normalized Gain	
	Coefficient	(p-value)	Coefficient	(p-value)
Combined	(1567 Observations, 234 Students)			
Control	0.064	(0.008)	0.158	(<0.001)
Fast	0.041	(0.199)	0.080	(0.015)
Medium	0.046	(0.161)	0.028	(0.409)
Slow	0.092	(0.004)	0.126	(<0.001)

Table 3: MLM for Learning Gain. AI assistant coefficients represent estimated average difference from Control.

gain as an observation and grouping these observations by student. The results for Combined dataset are shown in Table 3 and the left panel of Figure 5 (dark bars indicate statistically significant results).

In the statistical model, the Control group without AI is the reference category and demonstrated statistically significant learning gains from zero. For AI assistants, the coefficients in Table 3 indicate each group’s estimated average difference from Control. In the left panel of Figure 5, we present each group’s estimated average gains. Most notably, the Slow group showed significant increases in both relative and normalized gains.

• Slow’s Impact on Relative Gain

- **Winter 2025:** 280% relative improvement over Control ($p = 0.003$), increasing relative gain from 0.060 to 0.228 (+0.168).
- **Combined:** 140% relative improvement over Control ($p = 0.004$), increasing relative gain from 0.064 to 0.156 (+0.092).

• Slow’s Impact on Normalized Gain

- **Fall 2024:** 58% relative improvement over Control ($p = 0.041$), increasing normalized gain from 0.150 to 0.237 (+0.087).
- **Winter 2025:** 101% relative improvement over Control ($p = 0.001$), increasing normalized gain from 0.170 to 0.343 (+0.173).
- **Combined:** 80% relative improvement over Control ($p < 0.001$), increasing normalized gain from 0.158 to 0.284 (+0.126).

Our findings indicate that the slowest and most guided AI interactions provide the strongest support for student learning. Interestingly, the Fast group with rapid, unrestricted access to GPT-4o also

	Aligned (1141 obs, 233 students)		Challenging (426 obs, 229 students)	
Control	0.200	(p<0.001)	0.027	(p=0.579)
Fast	0.054	(p=0.148)	0.157	(p=0.021)
Medium	0.032	(p=0.431)	0.081	(p=0.197)
Slow	0.106	(p=0.007)	0.209	(p=0.001)

Table 4: Norm. Learning for Aligned vs. Challenging Labs

	Below Mean (677 obs, 208 students)		Above Mean (890 obs, 214 students)	
Control	0.259	(p<0.001)	-0.053	(p=0.259)
Fast	0.084	(p=0.099)	0.145	(p=0.023)
Medium	0.047	(p=0.363)	0.070	(p=0.276)
Slow	0.155	(p=0.002)	0.076	(p=0.226)

Table 5: Norm. Learning for Above/Below Mean

exhibited significantly higher normalized gains than the Control in the Combined dataset. However, the Medium group showed no statistical difference from the Control, suggesting that the moderate pace may be the least effective. These results imply that AI assistants should be intentionally slow to achieve measurable benefits compared to commercial LLMs. Meanwhile, medium-paced approaches featuring typical instructor preprompt elements such as tutor role-play, guardrails, usage categorization, and guiding questions appear insufficient to improve outcomes.

4.1.2 Diverging Impact of Pace. To better understand how both Slow and Fast models might yield substantial normalized learning gain, we examined the average pre-test scores. Recursion and Networks labs were especially challenging for students (mean ≤ 70). We categorized these as Challenging, and labeled the remaining labs as Aligned. As shown in Table 3, Slow demonstrated effectiveness in both lab types, whereas Fast only showed significant improvement within the Challenging subset.

Building upon Yeh et al. [35], we hypothesize that the Fast pace is helpful to students with more content mastery, whereas the Slow pace is helpful to students with less content mastery. To test this, we split the combined data into two sets (Below vs Above Mean pre-quiz scores). The estimated average difference from AI assistants to Control are shown in Table 5, and the estimated average normalized gain for each group is presented in the right panel of Figure 5.

Among students who scored below the mean on the pre-quiz, only the Slow model demonstrates a statistically significant positive effect (coefficient = 0.155, $p = 0.002$). By contrast, for students scoring above the mean, only the Fast model exhibits a statistically significant positive impact (coefficient = 0.145, $p = 0.023$). Splitting the data by the median yields the same pattern.

These results can be explained by theories in learning and psychology. Slower, guided questioning helps students who are mastering foundational concepts by scaffolding their learning within their zone of proximal development [34]. However, once students demonstrate higher proficiency (above mean pre-test scores), they may benefit more from rapid feedback instead of step-by-step guidance.

Aligning the mode of instruction (AI pace) with a student’s readiness fosters deeper comprehension. For less prepared learners, a more deliberate pace ensures steady growth at the right time. For

	Fall 2024	Winter 2025	Combined
Control	92.311 (<0.001)	113.190 (<0.001)	101.516 (<0.001)
Fast	-0.375 (0.920)	-1.439 (0.815)	-0.485 (0.886)
Medium	-0.787 (0.831)	-12.594 (0.079)	-6.063 (0.087)
Slow	-3.792 (0.305)	-9.559 (0.133)	-6.453 (0.060)

Table 6: Learning Retention. Control is the intercept.

	Challenging (322 obs, 225 students)		Above Mean (861 obs, 210 students)	
Control	106.148	(p<0.001)	102.382	(p<0.001)
Fast	-21.793	(p=0.015)	-9.748	(p=0.021)
Medium	-1.715	(p=0.822)	-4.331	(p=0.310)
Slow	-10.933	(p=0.158)	-3.931	(p=0.340)

Table 7: Retention for Challenging Labs and Above Mean

students with core foundational skills, moving quickly on advanced challenges can enhance motivation and engagement [3]. The Slow model’s Socratic approach promotes exploration and reflection, particularly benefiting those who need more time to process new concepts [15]. Meanwhile, the Fast model can alleviate cognitive load and frustration for advanced students facing difficult tasks [2].

Overall, prior research supports differentiated pacing based on initial skill levels. The Slow approach remains broadly effective and is especially crucial for those needing additional scaffolding, while the Fast approach helps higher-performing students progress at a rate aligned with their abilities. Across demographic groups, only Slow showed a significant positive effect (coefficient = 0.34, $p = 0.014$) on normalized learning among 14 late-enrolled students who struggled to catch up, reinforcing that a slower pace is especially beneficial for those with less prior knowledge.

4.1.3 Pairwise comparisons. After fitting each MLM, we conducted a Wald test to assess the overall effect of AI assistants. When that test indicated significance, we then performed model-based post-hoc pairwise comparisons with Bonferroni correction. In every instance where the Slow model was significant in the MLM, pairwise comparisons confirmed Slow’s positive impact on learning relative to Control. Slow also outperformed Medium for overall normalized learning. By contrast, pairwise comparisons revealed no statistically significant difference between Fast and Control.

4.2 Impact on Student Learning Retention (RQ2)

To examine how AI interaction pace affects learning retention, we measured the learning retention rate (expressed as a percentage) by comparing post-quiz scores to corresponding exam question scores. The exams consisted of multi-part questions, where each question covered one lab topic. We calculated the learning retention rate as:

$$R = \frac{S_{\text{exam}}}{S_{\text{post}}} \times 100\%$$

To examine learning retention, we used MLM regression on the Fall 2024, Winter 2025, and combined datasets. As shown in Table 6, no AI-supported group (Fast, Medium, or Slow) significantly differs from the Control group, since all p -values exceed 0.05.

To isolate differences between the Slow and Fast models in terms of learning retention, we split the Combined dataset into Aligned vs.

Challenging labs and Below Mean vs. Above Mean. Table 7 reports the statistically significant results. The Fast model shows notable learning retention degradation with a coefficient of -21.793 ($p=0.015$) for Challenging labs and -9.748 ($p=0.021$) for the Above Mean subset. These coefficients represent the average difference from Control. In contrast, there is no statistically significant degradation for Slow.

When viewed with the learning data, these findings show that the Slow model offers a clear advantage for student learning. Although the Fast model may provide short-term improvements, those gains largely dissipate when we look at knowledge retention.

5 Limitations and Threats to Validity

Apart from notes on the AI’s start page, students received no formal training on AI assistant usage. Our experiments measured only the learning that occurred during 80-minute structured labs with AI assistants. Despite spanning two terms, isolating AI interaction pacing’s impact on learning remains challenging, and unmeasured factors may have influenced the results. The slower pacing of the Slow LLM may be confounded with factors such as self-regulation, cognitive effort, and productive struggle, making it difficult to isolate the direct impact of pace on learning. Furthermore, the discussion on learning retention does not account for confounding variables such as activity duration, productive struggle, and cognitive effort. One Winter 2025 lab section was mistakenly assigned an incorrect model. Data from this section were excluded from the analysis. The reduced sample size may affect the statistical power and may introduce selection bias. This study was conducted at a Minority-Serving Institution within the United States. The educational practices, student demographics, and institutional culture of this university may not reflect those of other institutions.

6 Conclusion

This study highlights the crucial role of pacing in student-AI interactions in intro CS courses. Across two terms with 234 students, a slow-paced, Socratic-style AI assistant significantly improved learning and retention by promoting deeper engagement. Although advanced students see initial learning gains with a fast-paced model, reduced retention negates the short-term gains, creating a misleading sense of mastery. Surprisingly, the medium-paced assistant with typical instructor preprompts demonstrated no statistically significant improvements. While students can theoretically mimic slow-paced strategies with a faster model, doing so requires high levels of self-regulation and intentionality. Given the widespread use of fast-paced commercial AI tools, offering a slower-paced alternative could meaningfully enhance overall learning. Moving forward, we plan to refine fast-paced models for advanced learners to preserve learning gains without sacrificing retention. Additionally, we will analyze chat logs to measure cognitive effort, providing deeper insight into how pacing influences engagement and learning.

Acknowledgments

The authors thank Jennifer Wong-Ma and Tzu-Yi Chen for writing feedback, Alex Ihler for statistical analysis feedback, Mohamad Chaalan for sentiment analysis, Troy Yeh for retrieval augmented generation, Leo Chao and Candice Lu for proofreading.

References

- [1] Lorin W. Anderson and David R. Krathwohl. 2001. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York.
- [2] John D. Bransford, Ann L. Brown, and Rodney R. Cocking (Eds.). 2000. *How People Learn: Brain, Mind, Experience, and School*. National Academy Press, Washington, D.C.
- [3] Jerome S. Bruner. 1961. The Act of Discovery. *Harvard Educational Review* 31, 1 (1961), 21–32.
- [4] Garrett M. Fitzmaurice, Nan M. Laird, and James H. Ware. 2011. *Applied Longitudinal Analysis* (2 ed.). Wiley.
- [5] Hacer Güner and Erkan Er. 2025. AI in the classroom: Exploring students' interaction with ChatGPT in programming learning. *Education and Information Technologies* (2025), 1–27.
- [6] Richard R. Hake. 1998. Interactive-Engagement vs Traditional Methods: A Six-Thousand-Student Survey of Mechanics Test Data for Introductory Physics Courses. *American Journal of Physics* 66, 1 (1998), 64–74. doi:10.1119/1.18809
- [7] Thomas M. Haladyna, Steven M. Downing, and Michael C. Rodriguez. 2002. A Review of Multiple-Choice Item-Writing Guidelines for Classroom Assessment. *Applied Measurement in Education* 15, 3 (2002), 309–334.
- [8] IBM. 2024. IBM SPSS Statistics Documentation: Linear Mixed Models. <https://www.ibm.com/docs/en/spss-statistics/saas?topic=statistics-linear-mixed-models>
- [9] Sanna Järvelä and Allyson F. Hadwin. 2013. New Frontiers: Regulating Learning in CSCL. *Educational Psychologist* 48, 1 (2013), 25–39. arXiv:https://doi.org/10.1080/00461520.2012.748006 doi:10.1080/00461520.2012.748006
- [10] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 455, 23 pages. doi:10.1145/3544548.3580919
- [11] Majeed Kazemitabaar, Oliver Huang, Sangho Suh, Austin Z. Henley, and Tovi Grossman. 2024. Exploring the Design Space of Cognitive Engagement Techniques with AI-Generated Code for Enhanced Learning. arXiv:2410.08922 [cs.HC] <https://arxiv.org/abs/2410.08922>
- [12] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 650, 20 pages. doi:10.1145/3613904.3642773
- [13] Harsh Kumar, David M. Rothschild, Daniel G. Goldstein, and Jake Hofman. 2023. Math Education with Large Language Models: Peril or Promise? *SSRN Electronic Journal* (11 2023). doi:10.2139/ssrn.4641653 Available at SSRN: <https://ssrn.com/abstract=4641653> or <http://dx.doi.org/10.2139/ssrn.4641653>.
- [14] Mark Liffiton, Brad E. Sheese, Jaromir Savelka, and Paul Denny. 2024. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA.
- [15] Matthew Lipman. 2003. *Thinking in Education* (2nd ed.). Cambridge University Press, Cambridge, UK.
- [16] Meilu Liu, Lawrence Jun Zhang, and Christine Biebricher. 2024. Investigating students' cognitive processes in generative AI-assisted digital multimodal composing and traditional writing. *Computers & Education* 211 (2024), 104977. doi:10.1016/j.compedu.2023.104977
- [17] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA, 750–756. doi:10.1145/3626252.3630938
- [18] Rongxin Liu, Julianna Zhao, Benjamin Xu, Christopher Perez, Yuliia Zhukovets, and David J. Malan. 2025. Improving AI in CS50: Leveraging Human Feedback for Better Learning. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)* (Pittsburgh, PA, USA). ACM, New York, NY, USA, 7 pages. doi:10.1145/3641554.3701945
- [19] Francesca Lucchetti, Zixuan Wu, Arjun Guha, Molly Q Feldman, and Carolyn Jane Anderson. 2024. Substance Beats Style: Why Beginning Students Fail to Code with LLMs. arXiv:2410.19792 [cs.CY] <https://arxiv.org/abs/2410.19792>
- [20] Wenhan Lyu, Yimeng Wang, Tingting (Rachel) Chung, Yifan Sun, and Yixuan Zhang. 2024. Evaluating the Effectiveness of LLMs in Introductory Computer Science Education: A Semester-Long Field Study. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale* (Atlanta, GA, USA) (L@S '24). Association for Computing Machinery, New York, NY, USA, 63–74. doi:10.1145/3657604.3662036
- [21] CE McCoy. 2017. Understanding the Intention-to-treat Principle in Randomized Controlled Trials. *Western Journal of Emergency Medicine* 18, 6 (October 2017), 1075–1078. doi:10.5811/westjem.2017.8.35985 Epub 2017 Sep 18.
- [22] Ismael Villegas Molina, Audria Montalvo, Benjamin Ochoa, Paul Denny, and Leo Porter. 2024. Leveraging LLM Tutoring Systems for Non-Native English Speakers in Introductory CS Courses. arXiv:2411.02725 [cs.HC] <https://arxiv.org/abs/2411.02725>
- [23] Sydney Nguyen, Hannah McLean Babe, Yangtian Zi, Arjun Guha, Carolyn Jane Anderson, and Molly Q Feldman. 2024. How Beginning Programmers and Code LLMs (Mis)read Each Other. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA.
- [24] OpenAI. 2025. ChatGPT. <https://chatgpt.com> Accessed: 2025-01-18.
- [25] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education (ITICSE-WGR '23)*. Association for Computing Machinery, New York, NY, USA, 108–159.
- [26] James Prather, Brent N Reeves, Juho Leinonen, Stephen MacNeil, Arisosa S Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. 2024. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1* (Melbourne, VIC, Australia) (ICER '24). Association for Computing Machinery, New York, NY, USA, 469–486. doi:10.1145/3632620.3671116
- [27] Nishat Raihan, Mohammed Latif Siddiqi, Joanna C. S. Santos, and Marcos Zampieri. 2024. Large Language Models in Computer Science Education: A Systematic Literature Review. arXiv:2410.16349 [cs.LG] <https://arxiv.org/abs/2410.16349>
- [28] Anshul Shah, Anya Chernova, Elena Tomson, Leo Porter, William G. Griswold, and Adalbert Gerald Soosai Raj. 2025. Students' Use of GitHub Copilot for Working with Large Code Bases. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) (SIGCSE TS 2025). Association for Computing Machinery, New York, NY, USA, 1050–1056. doi:10.1145/3641554.3701800
- [29] Brad Sheese, Mark Liffiton, Jaromir Savelka, and Paul Denny. 2024. Patterns of Student Help-Seeking When Using a Large Language Model-Powered Programming Assistant. In *Proceedings of the 26th Australasian Computing Education Conference* (Sydney, NSW, Australia) (ACE '24). Association for Computing Machinery, New York, NY, USA, 49–57. doi:10.1145/3636243.3636249
- [30] Esther Shein. 2024. The Impact of AI on Computer Science Education. *Commun. ACM* 67, 9 (Aug. 2024), 13–15. doi:10.1145/3673428
- [31] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. 2024. The Metacognitive Demands and Opportunities of Generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 680, 24 pages. doi:10.1145/3613904.3642902
- [32] Toyin Tofade, Jamie Elsner, and Stuart T Haines. 2013. Best Practice Strategies for Effective Use of Questions as a Teaching Tool. *American Journal of Pharmaceutical Education* 77, 7 (2013), 155. doi:10.5688/ajpe777155
- [33] University of Pittsburgh Center for Teaching and Learning. [n. d.]. Teaching Through Questioning: Socratic No More? <https://teaching.pitt.edu/resources/teaching-through-questioning-socratic-no-more/>.
- [34] Lev S. Vygotsky. 1978. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA.
- [35] Thomas Y. Yeh, Karena Tran, Ge Gao, Tyler Yu, Wai On Fong, and Tzu-Yi Chen. 2025. Bridging Novice Programmers and LLMs with Interactivity. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) (SIGCSE TS 2025). Association for Computing Machinery, New York, NY, USA, 1295–1301. doi:10.1145/3641554.3701867
- [36] Hao Yu. 2023. Reflection on whether Chat GPT should be banned by academia from the perspective of education and teaching. *Frontiers in Psychology* 14 (2023).